

Academia de Studii Economice, București

Facultatea de Cibernetică, Statistică și Informatică Economică

Specializarea: Informatică Economică

Proiect

Proiectare și programare orientate obiect

Aplicație Java pentru gestiunea comenzilor unui restaurant

Covrig Eduard-Gabriel

E-BUSINESS

1. Descrierea Generală și Arhitectura Aplicației

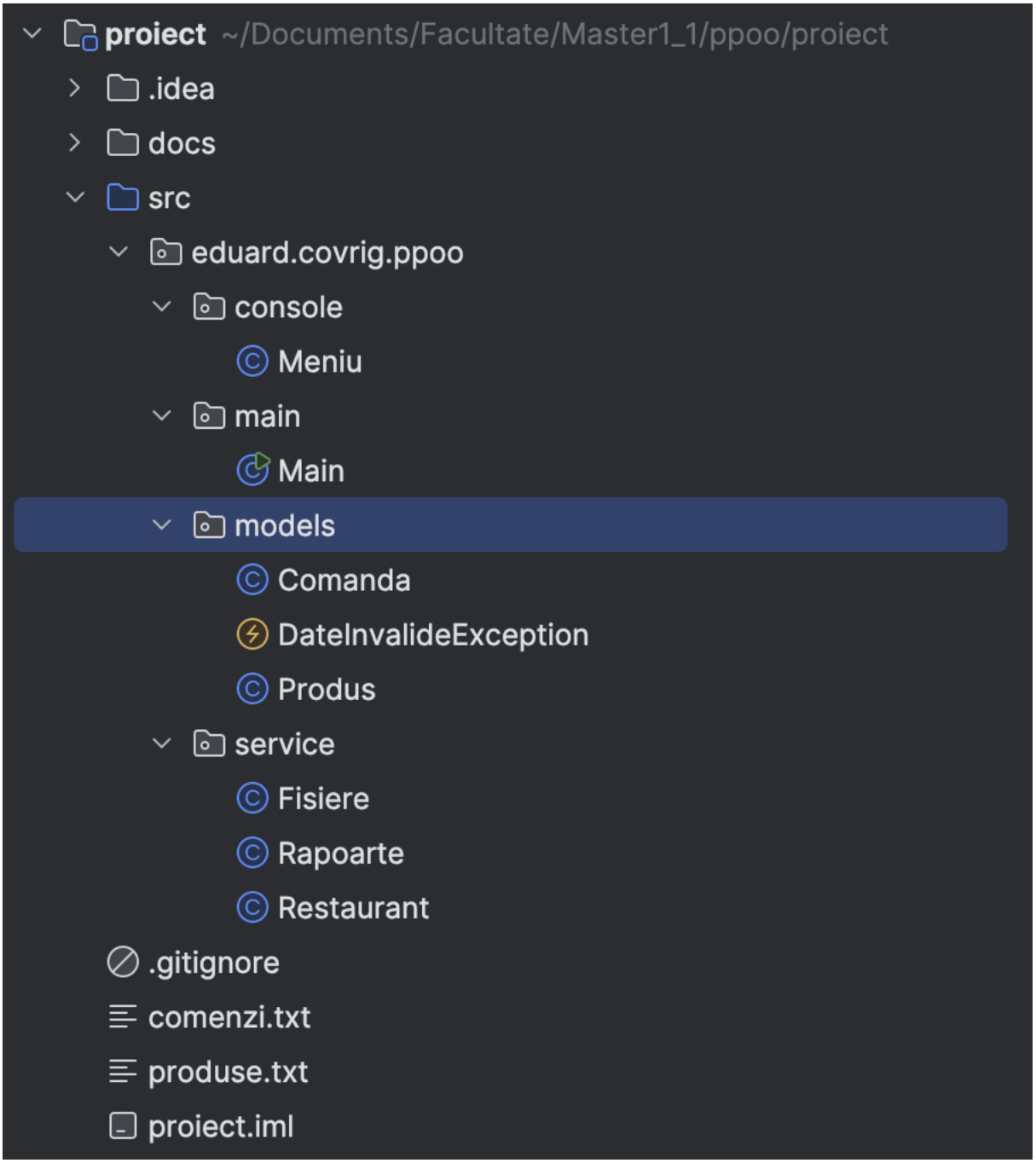
Această aplicație reprezintă un sistem software modular, dezvoltat în limbajul Java, conceput pentru optimizarea și automatizarea fluxurilor operaționale dintr-un restaurant. Soluția acoperă întregulciclu de viață al datelor specifice unui astfel de business: catalogul de produse, preluarea și gestiunea comenzilor, persistența automată în fișiere de tip text, precum și generarea de rapoarte statistice și financiare avansate.

Componente Arhitecturale Principale:

- **Gestiunea Catalogului de Produse:** Permite operațiuni CRUD (Creare, Citire, Ștergere) pentru articole structurate pe categorii (aperitive, supe, feluri principale, deserturi, băuturi), validând integritatea atributelor (preț, gramaj, identificator unic).
- **Sistemul de Procesare a Comenzilor:** Asociază comenzile cu marcaje temporale precise (LocalDateTime), gestionează cantitățile asociate fiecărui produs prin structuri de date mapate și calculează automat valoarea totală.
- **Mecanismul de Persistență:** Asigură încărcarea și salvarea stării aplicației la inițializare și închidere, utilizând fișiere text structurate (produse.txt și comenzi.txt).
- **Subsistemul de Raportare Analitică:** Centralizează indicatori de performanță (cifră de afaceri pe zi, volumul comenzilor pe intervale orare, topul produselor comerciale și distribuția veniturilor pe zilele săptămânii), oferind opțiunea de export într-un fișier dedicat de tip raport.

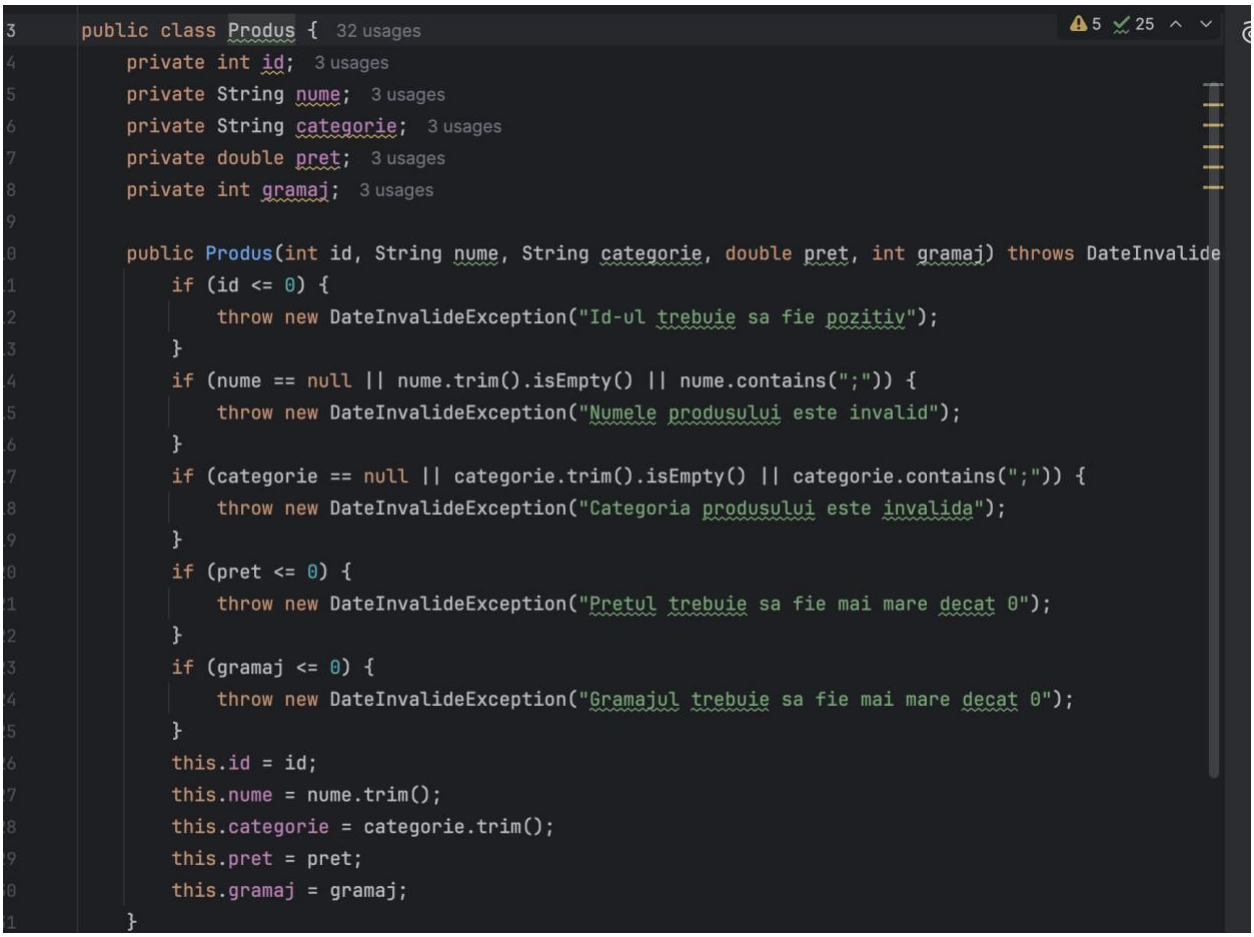
2. Pachetul eduard.covrig.ppoo.models

Acest pachet definesc entitățile fundamentale ale domeniului de business și mecanismul de tratare a excepțiilor.



Clasa Produs

Modelează un articol din meniul restaurantului. Implementează validări stricte la nivelul constructorului pentru a preveni introducerea de date eronate (valori negative pentru preț/gramaj, identificatori nuli sau caractere speciale care ar compromite formatul fișierelor de persistență).



```java
public class Produs {  32 usages
    private int id;  3 usages
    private String nume;  3 usages
    private String categorie;  3 usages
    private double pret;  3 usages
    private int gramaj;  3 usages

    public Produs(int id, String nume, String categorie, double pret, int gramaj) throws DateInvalide
        if (id <= 0) {
            throw new DateInvalideException("Id-ul trebuie sa fie pozitiv");
        }
        if (nume == null || nume.trim().isEmpty() || nume.contains(";")) {
            throw new DateInvalideException("Numele produsului este invalid");
        }
        if (categorie == null || categorie.trim().isEmpty() || categorie.contains(";")) {
            throw new DateInvalideException("Categoria produsului este invalida");
        }
        if (pret <= 0) {
            throw new DateInvalideException("Pretul trebuie sa fie mai mare decat 0");
        }
        if (gramaj <= 0) {
            throw new DateInvalideException("Gramajul trebuie sa fie mai mare decat 0");
        }
        this.id = id;
        this.nume = nume.trim();
        this.categorie = categorie.trim();
        this.pret = pret;
        this.gramaj = gramaj;
    }
```

Clasa Comanda

Encapsulează datele unei comenzi efectuate de un client: identificatorul numeric unic, marca temporală a plasării (LocalDateTime) și o colecție de tip Map<Produs, Integer> care realizează asocierea cantitativă între articolele selectate și starea comenzii.



```java
public class Comanda {  31 usages
    private int id;  3 usages
    private LocalDateTime data;  3 usages
    // produsul comandat si cantitatea lui
    private Map<Produs, Integer> produse = new LinkedHashMap<>();  4 usages

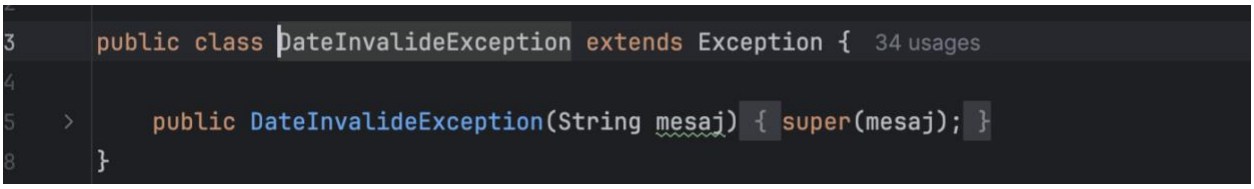
    public Comanda(int id, LocalDateTime data) {  2 usages
        this.id = id;
        this.data = data;
    }

    public void adauga(Produs produs, int cantitate) throws DateInvalideException {  2 usages
        if (cantitate <= 0) {
            throw new DateInvalideException("Cantitatea trebuie sa fie mai mare decat 0");
        }
        // daca produsul e deja in comanda, adunam cantitatea
        produse.put(produs, produse.getOrDefault(produs, defaultValue: 0) + cantitate);
    }

    public double getTotal() {  5 usages
        double total = 0;
        for (Map.Entry<Produs, Integer> e : produse.entrySet()) {
            total += e.getKey().getPret() * e.getValue();
        }
        return total;
    }
```

Clasa DateInvalideException

O excepție personalizată, extinsă din clasa Exception, utilizată pentru semnalarea erorilor de validare a datelor de intrare sau a încălcărilor regulilor de business pe parcursul execuției.



```java
public class DateInvalideException extends Exception {  34 usages

    public DateInvalideException(String mesaj) { super(mesaj); }
}
```

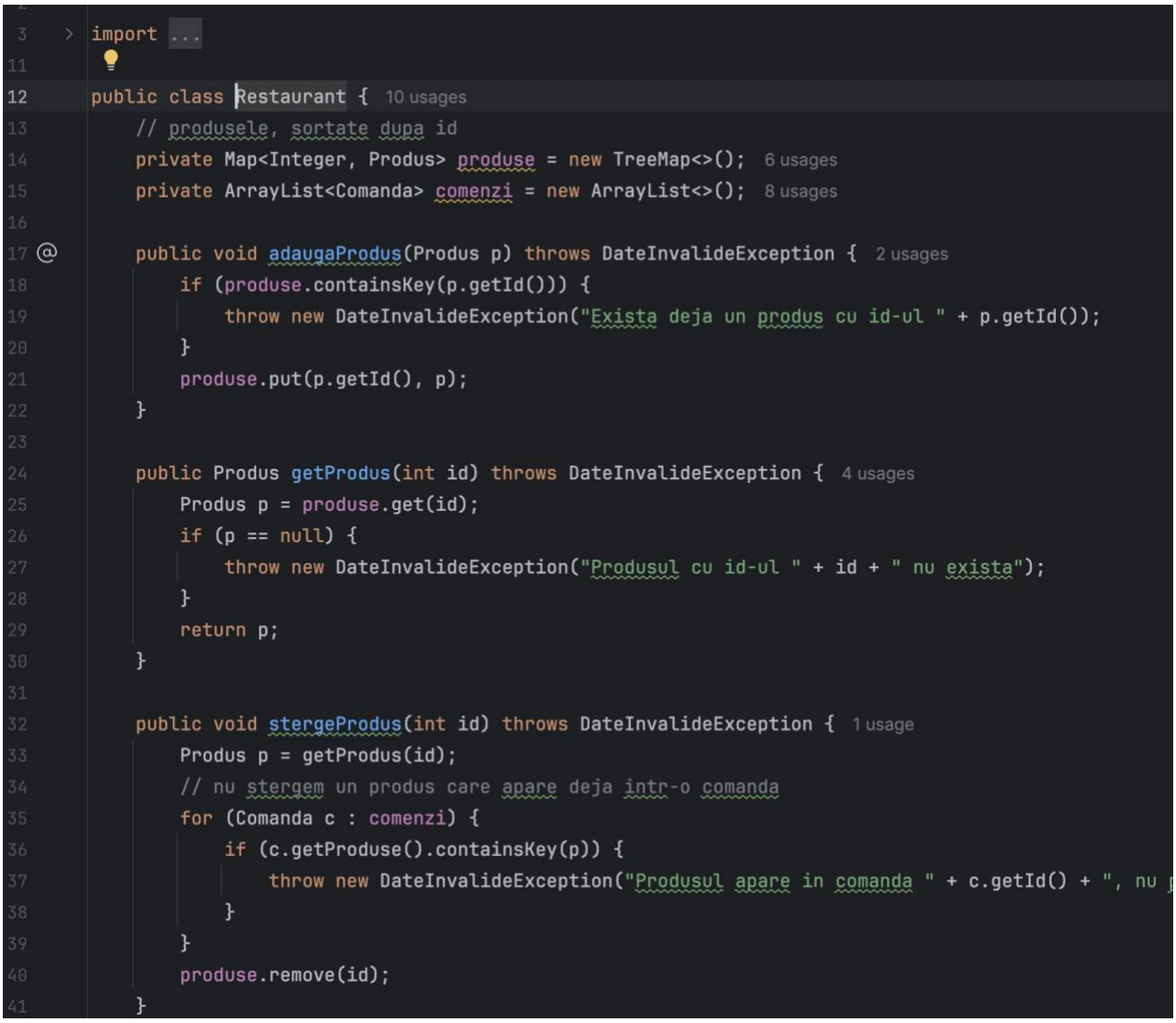
3. Pachetul eduard.covrig.ppoo.service

Reprezintă nucleul logic al aplicației (*Business Logic*), fiind compus din clase specializate pe servicii distincte.

Clasa Restaurant

Funcționează ca entitate centrală de agregare a stării în memorie. Utilizează un TreeMap pentru menținerea produselor ordonate după ID și un ArrayList pentru stocarea cronologică a comenzilor.

Implementează constrângeri de integritate referențială (de exemplu, interzicerea ștergerii unui produs care se regăsește într-o comandă activă).



```java
import ...

public class Restaurant {  10 usages
    // produsele, sortate dupa id
    private Map<Integer, Produs> produse = new TreeMap<>();  6 usages
    private ArrayList<Comanda> comenzi = new ArrayList<>();  8 usages

    public void adaugaProdus(Produs p) throws DateInvalideException {  2 usages
        if (produse.containsKey(p.getId())) {
            throw new DateInvalideException("Exista deja un produs cu id-ul " + p.getId());
        }
        produse.put(p.getId(), p);
    }

    public Produs getProdus(int id) throws DateInvalideException {  4 usages
        Produs p = produse.get(id);
        if (p == null) {
            throw new DateInvalideException("Produsul cu id-ul " + id + " nu exista");
        }
        return p;
    }

    public void stergeProdus(int id) throws DateInvalideException {  1 usage
        Produs p = getProdus(id);
        // nu stergem un produs care apare deja intr-o comanda
        for (Comanda c : comenzi) {
            if (c.getProduse().containsKey(p)) {
                throw new DateInvalideException("Produsul apare in comanda " + c.getId() + ", nu
            }
        }
        produse.remove(id);
    }
```

Clasa Rapoarte

Procesează colecțiile de date gestionate de clasa Restaurant pentru a extrage indicatori statistici și ierarhii de vânzări. De asemenea, gestionează formatarea și scrierea rapoartelor textuale complexe în fișierul de ieșire.



```java
import ...

public class Rapoarte {  3 usages
    private Restaurant restaurant;  12 usages
    private String[] zile = {"Luni", "Marti", "Miercuri", "Joi", "Vineri", "Sambata", "Duminica"};

    public Rapoarte(Restaurant restaurant) { this.restaurant = restaurant; }

    public double totalZi(LocalDate zi) {  1 usage
        double suma = 0;
        for (Comanda c : restaurant.comenziDinZi(zi)) {
            suma += c.getTotal();
        }
        return suma;
    }

    public int nrComenzi(LocalDate de, LocalDate pana) throws DateInvalideException {  1 usage
        if (de.isAfter(pana)) {
            throw new DateInvalideException("Data de inceput este dupa data de sfarsit");
        }
        int nr = 0;
        for (Comanda c : restaurant.getComenzi()) {
            LocalDate zi = c.getData().toLocalDate();
            if (!zi.isBefore(de) && !zi.isAfter(pana)) {
                nr++;
            }
        }
        return nr;
    }

    public List<Map.Entry<Produs, Integer>> topProduse(int n) {  2 usages
        Map<Produs, Integer> cantitati = new HashMap<>();
```

Clasa Fisiere

Gestionează fluxurile deintrare/ieșire (*I/O*) prin intermediul claselor BufferedReader și PrintWriter, asigurând citirea și scrierea robustă a datelor din fișierele text. Include mecanisme de filtrare a liniilor corupte sau nule pentru a asigura continuitatea execuției.



```
public class Fisiere {  3 usages
    private static final String FISIER_PRODUSE = "produse.txt";  3 usages
    private static final String FISIER_COMENZI = "comenzi.txt";  3 usages

    public static void incarca(Restaurant r) throws IOException {  1 usage
        // produsele se citesc primele, comenzile au nevoie de ele
        File fp = new File(FISIER_PRODUSE);
        if (fp.exists()) {
            try (BufferedReader br = new BufferedReader(new FileReader(fp))) {
                String linie;
                while ((linie = br.readLine()) != null) {
                    if (linie.trim().isEmpty()) {
                        continue;
                    }
                    try {
                        String[] p = linie.split( regex: ";");
                        r.adaugaProdus(new Produs(Integer.parseInt(p[0]), p[1], p[2],
                                Double.parseDouble(p[3]), Integer.parseInt(p[4])));
                    } catch (NumberFormatException | ArrayIndexOutOfBoundsException | DateInvalideE
                        System.out.println("Linie ignorata din " + FISIER_PRODUSE + ": " + linie);
                    }
                }
            }
        }

        File fc = new File(FISIER_COMENZI);
        if (fc.exists()) {
            try (BufferedReader br = new BufferedReader(new FileReader(fc))) {
                String linie;
                while ((linie = br.readLine()) != null) {
                    if (linie.trim().isEmpty()) {
                        continue;
                    }
```

4. Pachetul eduard.covrig.ppoo.console si Main

Clasa Meniu

Oferă interfața de tip linie de comandă (*CLI*), facilitând interacțiunea utilizatorului prin intermediul fluxului standard. Integrează blocuri de tratare a excepțiilor pentru a gestiona erorile de conversie a tipurilor de date sau opțiunile invalide introduse de utilizator.

```java
import ...

public class Meniu {  2 usages
    private Restaurant restaurant;  13 usages
    private Rapoarte rapoarte;  11 usages
    private Scanner sc = new Scanner(System.in);  3 usages

    public Meniu(Restaurant restaurant) {  1 usage
        this.restaurant = restaurant;
        this.rapoarte = new Rapoarte(restaurant);
    }

    public void start() {  1 usage
        int optiune = -1;
        while (optiune != 0) {
            System.out.println("\n===== MENIU RESTAURANT =====");
            System.out.println("1. Afiseaza produsele");
            System.out.println("2. Adauga produs");
            System.out.println("3. Sterge produs");
            System.out.println("4. Comanda noua");
            System.out.println("5. Afiseaza toate comenzile");
            System.out.println("6. Detalii comanda");
            System.out.println("7. Sterge comanda");
            System.out.println("8. Rapoarte");
            System.out.println("0. Iesire (salveaza datele)");
            try {
                optiune = citesteInt( mesaj: "Alege optiunea: ");
                switch (optiune) {
                    case 1:
                        afiseazaProduse();
                        break;
                    case 2:
```

```
/Users/eduardcovrig/Library/Java/JavaVirtualMachines/jdk-21.0.2.jdk/Contents/Home/bin/java -javaagent:/A
Date incarcate: 15 produse, 29 comenzi

===== MENIU RESTAURANT =====
1. Afiseaza produsele
2. Adauga produs
3. Sterge produs
4. Comanda noua
5. Afiseaza toate comenzile
6. Detalii comanda
7. Sterge comanda
8. Rapoarte
0. Iesire (salveaza datele)
Alege optiunea:
```

Clasa Main

Reprezintă punctului de intrare în aplicație (main). Coordonează secvența de inițializare prin încărcarea datelor persistente, instanțierea interfeței interactive și asigurarea salvării automate a stării curente la închiderea programului.

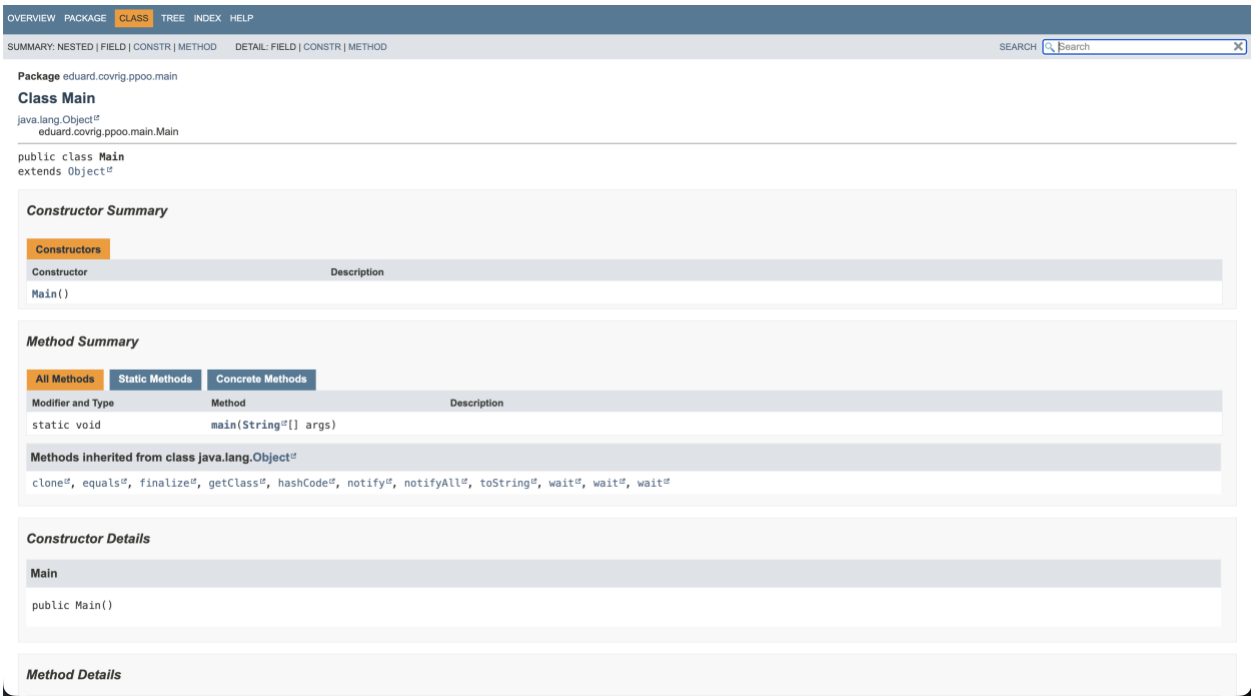
```java
public class Main {
    public static void main(String[] args) {
        Restaurant restaurant = new Restaurant();

        // la pornire incarcam datele din fisiere
        try {
            Fisiere.incarca(restaurant);
        } catch (IOException e) {
            System.out.println("Eroare la citirea fisierelor: " + e.getMessage());
            return;
        }
        System.out.println("Date incarcate: " + restaurant.getProduse().size() + " produse, "
                + restaurant.getComenzi().size() + " comenzi");

        new Meniu(restaurant).start();

        // la inchidere salvam tot in fisiere
        try {
            Fisiere.salveaza(restaurant);
            System.out.println("Datele au fost salvate. La revedere!");
        } catch (IOException e) {
            System.out.println("Eroare la salvarea fisierelor: " + e.getMessage());
        }
    }
}
```

Java docs:



OVERVIEW PACKAGE CLASS TREE INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH Search

Package eduard.covrig.ppoo.main

Class Main

java.lang.Object
eduard.covrig.ppoo.main.Main

```
public class Main
extends Object
```

Constructor Summary

Constructors

Constructor	Description
Main()	

Method Summary

All Methods | Static Methods | Concrete Methods

Modifier and Type	Method	Description
static void	main(String[] args)	

Methods inherited from class java.lang.Object

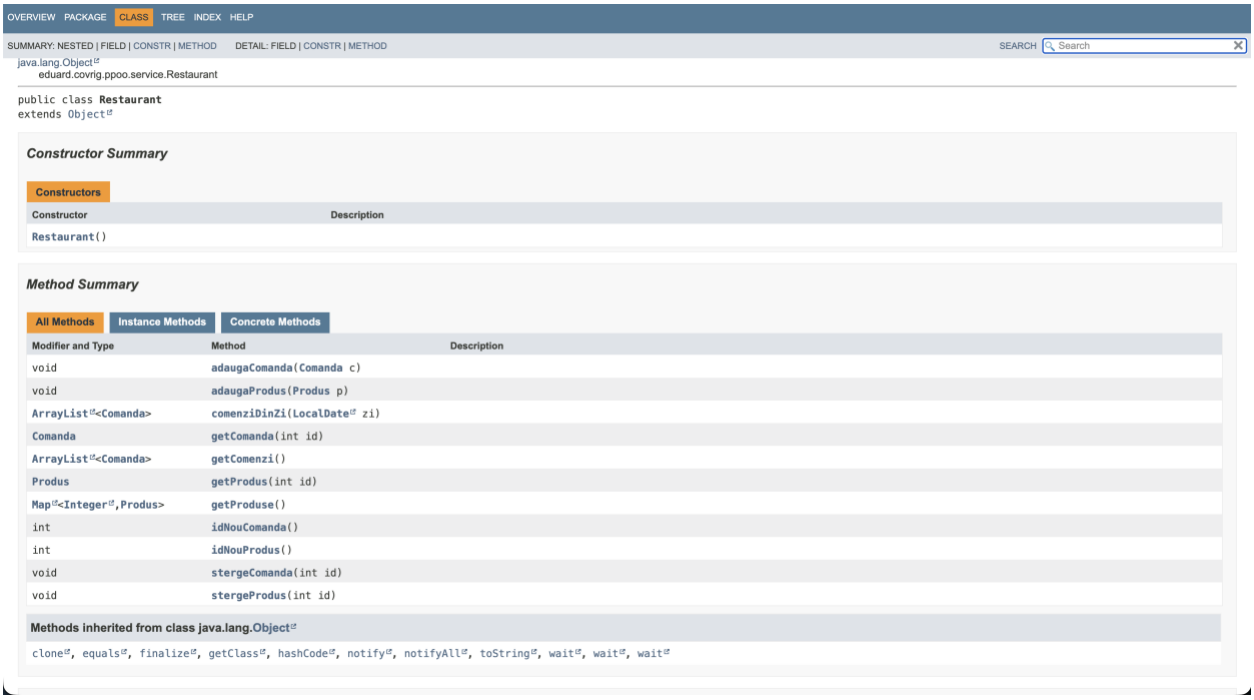
clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

Main

```
public Main()
```

Method Details



OVERVIEW PACKAGE CLASS TREE INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH Search

java.lang.Object
eduard.covrig.ppoo.service.Restaurant

```
public class Restaurant
extends Object
```

Constructor Summary

Constructors

Constructor	Description
Restaurant()	

Method Summary

All Methods | Instance Methods | Concrete Methods

Modifier and Type	Method	Description
void	adaugaComanda(Comanda c)	
void	adaugaProdus(Produs p)	
ArrayList<Comanda>	comenziDinZi(LocalDate zi)	
Comanda	getComanda(int id)	
ArrayList<Comanda>	getComenzi()	
Produs	getProdus(int id)	
Map<Integer,Produs>	getProduse()	
int	idNouComanda()	
int	idNouProdus()	
void	stergeComanda(int id)	
void	stergeProdus(int id)	

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Package eduard.covrig.ppoo.service

Class Fisiere

java.lang.Object
eduard.covrig.ppoo.service.Fisiere

public class **Fisiere**
extends Object

Constructor Summary

Constructors

Constructor	Description
Fisiere()	

Method Summary

All Methods | **Static Methods** | **Concrete Methods**

Modifier and Type	Method	Description
static void	**incarca**(**Restaurant** r)	
static void	**salveaza**(**Restaurant** r)	

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

Fisiere

```
public Fisiere()
```

Constructor Details

Fisiere

```
public Fisiere()
```

Method Details

incarca

```
public static void incarca(Restaurant r)
                    throws IOException
```

Throws:
IOException

salveaza

```
public static void salveaza(Restaurant r)
                     throws IOException
```

Throws:
IOException